PCAP Nikto Replay Simulation

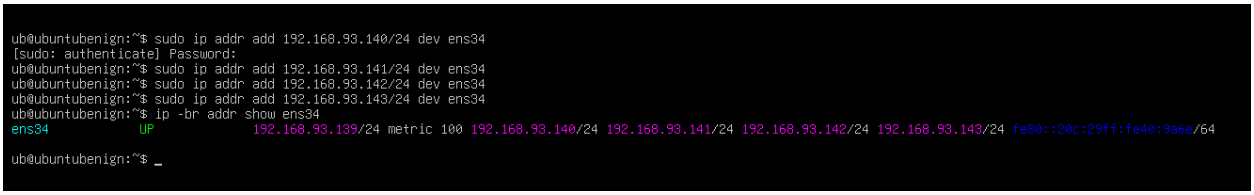
Date	August 11, 2026

Step 1 — give Ubuntu-Benign 5 real addresses:

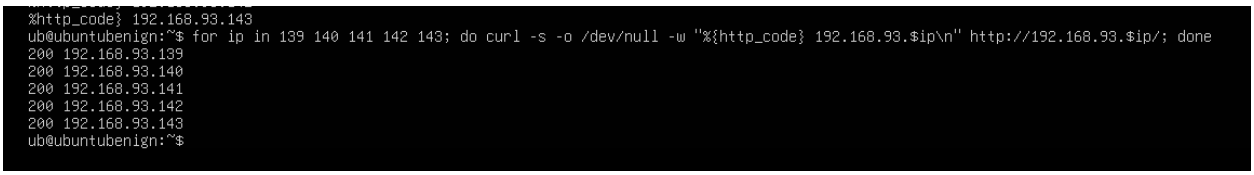
```
sudo ip addr add 192.168.93.140/24 dev ens34
sudo ip addr add 192.168.93.141/24 dev ens34
sudo ip addr add 192.168.93.142/24 dev ens34
sudo ip addr add 192.168.93.143/24 dev ens34
```

What this does: Linux lets a single network interface hold multiple IP addresses simultaneously (this is called adding "secondary" or "alias" addresses). `ens34` keeps its original `192.168.93.139` and gains four more. To anything on the network, it now genuinely looks like there are 5 separate hosts at those 5 addresses — but it's really one machine, one Apache instance, listening on all of them at once (since Apache by default binds to `0.0.0.0`, meaning "every address this machine has," not just one).

We can see all five addresses.



Confirm Apache genuinely answers on every one of them (this is the part that actually matters — if any of these fail, that "victim" won't be able to complete a real handshake and you'll hit bug #6's symptom again for that specific IP):



Step 2 — generate 15 attacker-vs-5-victim traffic (Kali VM):

```
#!/bin/bash
# generate_15_vs_5.sh
INPUT="sample_attack.pcap"
WORKDIR="mh_tmp"
OUTPUT="15_vs_5_simulation.pcap"
SURICATA_MAC="00:0c:29:9b:a0:64"

mkdir -p "$WORKDIR"
rm -f "$WORKDIR"/*.pcap

ATTACKERS=(10.0.0.1 10.0.0.2 10.0.0.3 10.0.0.4 10.0.0.5 10.0.
0.6 10.0.0.7 10.0.0.8 10.0.0.9 10.0.0.10 10.0.0.11 10.0.0.12
10.0.0.13 10.0.0.14 10.0.0.15)
VICTIMS=(192.168.93.139 192.168.93.140 192.168.93.141 192.16
8.93.142 192.168.93.143)
REAL_CLIENT="192.168.93.130"
REAL_VICTIM="192.168.93.139"

i=0
for atk in "${ATTACKERS[@]}"; do
    vic=${VICTIMS[$(( i % 5 ))]}
    echo "[+] Building attacker $atk -> victim $vic"
    sudo tcprewrite \
      --infile="$INPUT" \
      --outfile="$WORKDIR/atk_${i}.pcap" \
      --srcipmap=${REAL_CLIENT}/32:${atk}/32,${REAL_VICTIM}/3
2:${vic}/32 \
      --dstipmap=${REAL_CLIENT}/32:${atk}/32,${REAL_VICTIM}/3
2:${vic}/32 \
      --enet-dmac="$SURICATA_MAC" \
      --fixcsum --fixhdrlen
    i=$((i+1))
done
```

```
echo "[+] Merging 15 attacker streams into one pcap..."
mergecap -w "$OUTPUT" "$WORKDIR"/atk_*.pcap

echo "[+] Verifying topology:"
tshark -r "$OUTPUT" -T fields -e ip.src -e ip.dst | sort -u
```

15-vs-5 Script Notes

Goal: Take one real Nikto attack capture and turn it into 15 fake attackers hitting 5 real victims without hitting the "one-way traffic" bug again.

- `ATTACKERS` — a list of 15 made-up IP addresses. They don't need to be real or actually exist on the network, they're just fake names we're pasting onto the traffic.
- `VICTIMS` — the 5 real IP addresses we added to Ubuntu-Benign in Step 1.
- `REAL_CLIENT` / `REAL_VICTIM` — the two real IP addresses that are actually in the original capture: Kali (the real attacker) and Ubuntu-Benign (the real victim). These are just the two "real" hosts we want to disguise.
- `-srcipmap=...`
  This tells `tcprewrite`: "any time you see the real client OR the real victim show up as the *sender* of a packet, replace it with its fake name." This covers both the client sending its attack **and** the victim sending its reply back.
- `-dstipmap=...`
  Same exact swap, but for whenever the real client or real victim shows up as the *receiver* of a packet. **This is the actual fix.** Before, we were only doing this for the victim's side, so replies going back to the real client never got disguised — which broke the connection tracking. Now both people, in both roles (sender and receiver), always get the same fake name.
- **What this gives us:** each fake attacker's file now has the client fully disguised everywhere it appears, and the victim fully disguised everywhere it appears — so the request and its reply still look like one matching conversation to Suricata, instead of two unrelated pieces.

- `mergecap` — glues all 15 separate files into one, sorted by time, so it looks like 15 attackers hitting 5 victims around the same time not one attacker repeating the same attack 15 times in a row.
- **Final check with** `tshark` — for each of the 15 fake attackers, we should now see **two lines**: one for the attack going out, one for the reply coming back. That's 30 lines total. If we only see 15 lines (no replies), the bug is back — stop and check before going further.

▼ 15 fake attackers, 5 real victims

Walking through what actually happens, in order

1. You start with one real recording. `sample_attack.pcap` is a real Nikto scan: real Kali (`192.168.93.130`) attacking real Ubuntu-Benign (`192.168.93.139`). One conversation, two real machines.

2. The script takes that single recording and makes 15 disguised copies of it. For each of the 15 loop iterations, it runs `tcprewrite` on the *same original file*, but each time:

- Replaces the real Kali IP with a different fake IP (`10.0.0.1`, `10.0.0.2`, ... up through `10.0.0.15`) — these fake IPs don't need to exist anywhere on the real network. They're just labels pasted onto the traffic.
- Replaces the real Ubuntu-Benign IP with one of the 5 *real* victim addresses you set up in Step 1 (`.139`, `.140`, `.141`, `.142`, `.143` — cycling through them so the load spreads across all 5).

So after this step, you have 15 separate pcap files (`atk_0.pcap` through `atk_14.pcap`), each one being **the exact same attack content**, just wearing a different fake attacker "mask" and aimed at one of your 5 real victims.

3. `mergecap` **glues all 15 files into one.** It also sorts everything by timestamp, so when it's replayed, it looks like all 15 attacks are happening around the same time rather than one after another in a slow sequence.

4. That merged file gets replayed onto the network. `tcpreplay` sends it out — real Ethernet frames hitting the wire, addressed as if they're coming from 15 different attacker machines.

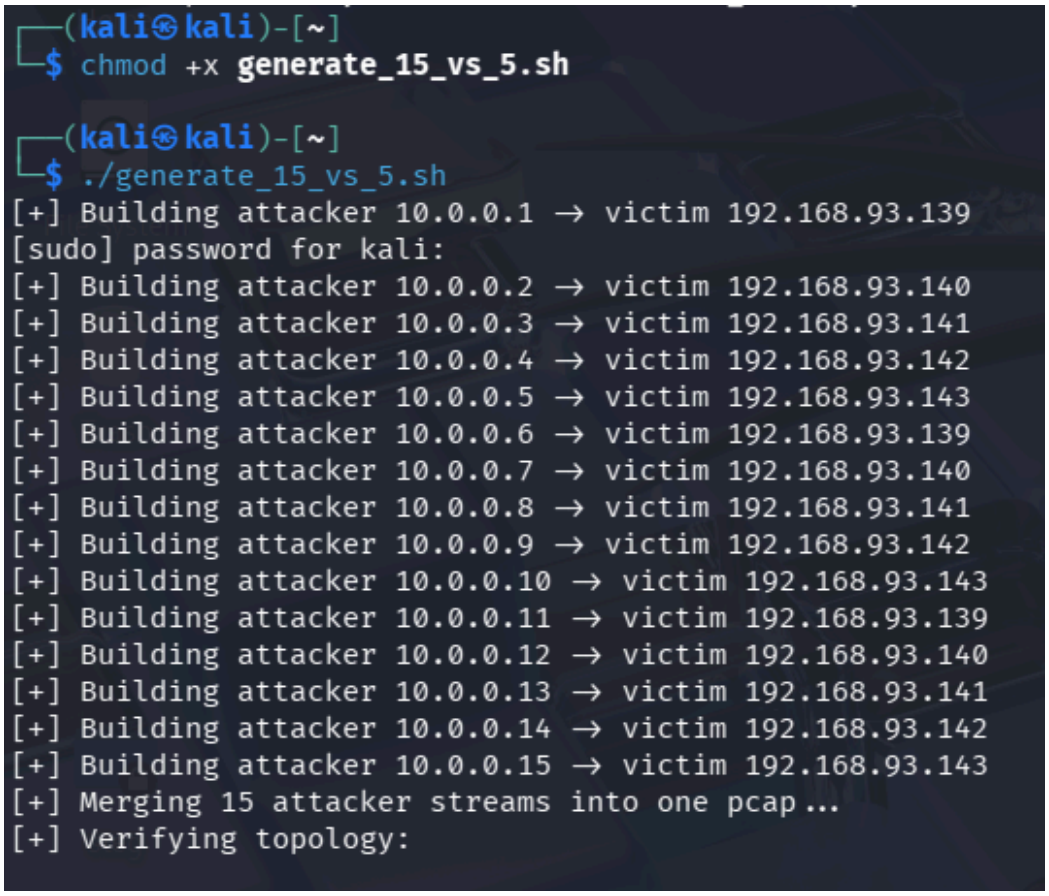
5. Ubuntu-Benign — because it's genuinely multi-homed now (owns all 5 real IP addresses on one interface) — actually receives and responds to all 5 "different" victim addresses, because it's really just one Apache server quietly answering on every address it owns.

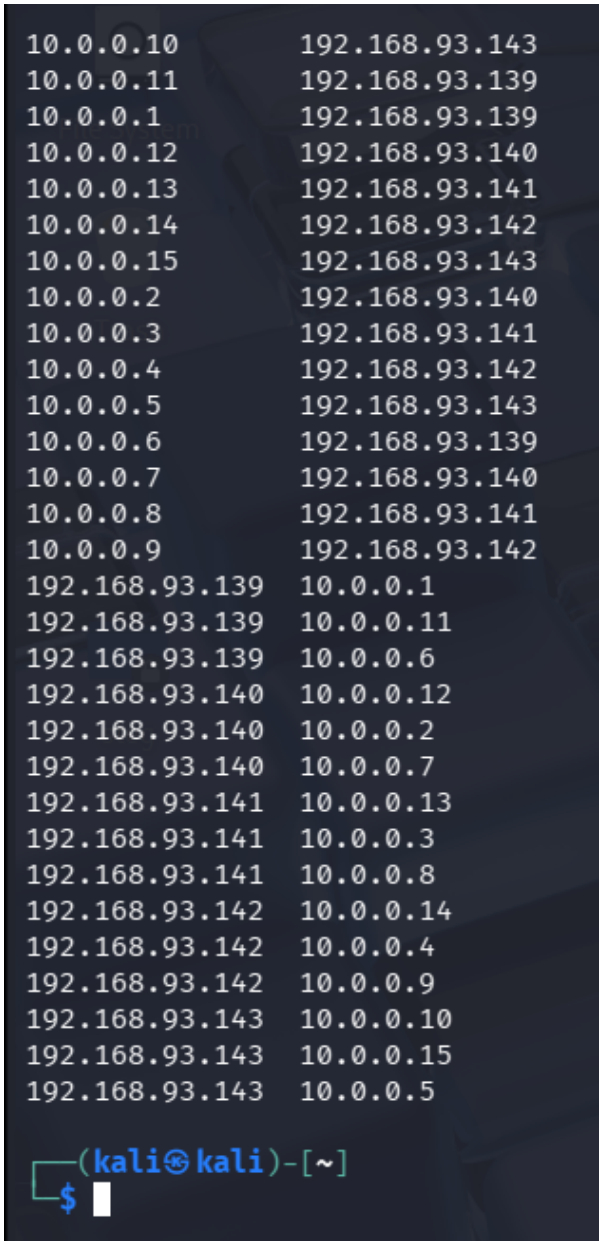
6. Suricata, watching the wire, sees what looks like 15 separate real attackers hitting 5 separate real victims — and generates alerts for each one, tagged with the fake attacker IP and the real victim IP.

7. Your dashboard shows all of this — 15 distinct attacker IPs, 5 distinct victim IPs, a realistic-looking multi-host attack, built entirely from one original single-attacker recording.

The short version

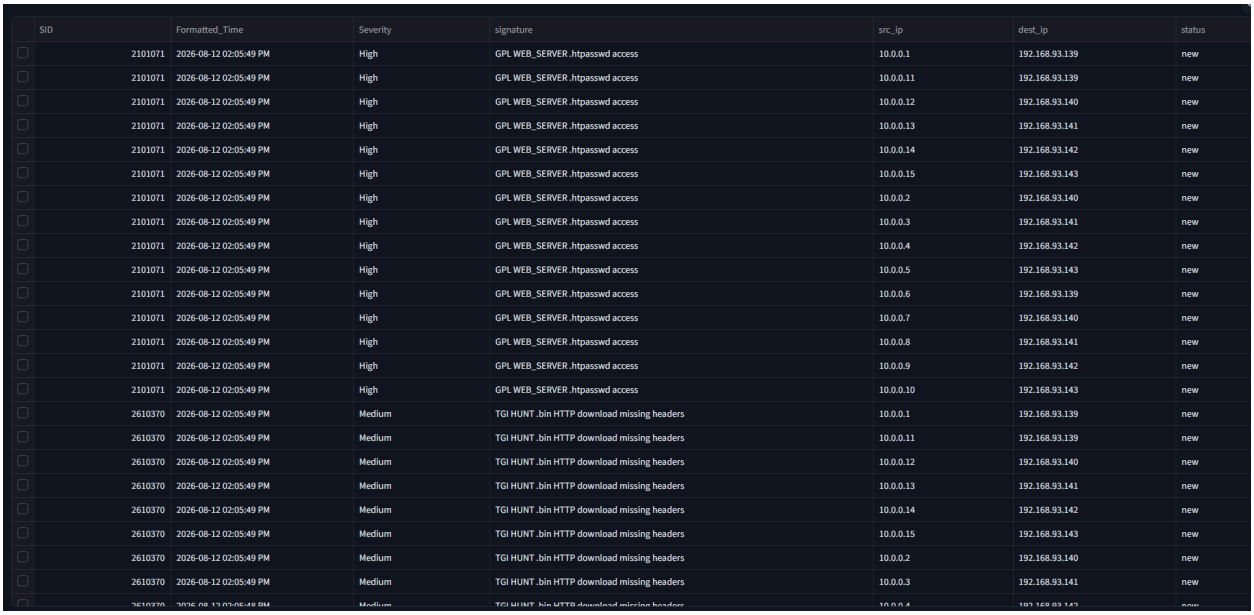
One real attack → copied and relabeled 15 times with fake attacker names → each copy aimed at one of 5 genuinely-real victim addresses → merged into one file → replayed → Suricata detects all 15 as if they were real, independent attackers.


┌──(kali㉿kali)-[~]
└─$ chmod +x generate_15_vs_5.sh
┌──(kali㉿kali)-[~]
└─$ ./generate_15_vs_5.sh
[+] Building attacker 10.0.0.1 → victim 192.168.93.139
[sudo] password for kali:
[+] Building attacker 10.0.0.2 → victim 192.168.93.140
[+] Building attacker 10.0.0.3 → victim 192.168.93.141
[+] Building attacker 10.0.0.4 → victim 192.168.93.142
[+] Building attacker 10.0.0.5 → victim 192.168.93.143
[+] Building attacker 10.0.0.6 → victim 192.168.93.139
[+] Building attacker 10.0.0.7 → victim 192.168.93.140
[+] Building attacker 10.0.0.8 → victim 192.168.93.141
[+] Building attacker 10.0.0.9 → victim 192.168.93.142
[+] Building attacker 10.0.0.10 → victim 192.168.93.143
[+] Building attacker 10.0.0.11 → victim 192.168.93.139
[+] Building attacker 10.0.0.12 → victim 192.168.93.140
[+] Building attacker 10.0.0.13 → victim 192.168.93.141
[+] Building attacker 10.0.0.14 → victim 192.168.93.142
[+] Building attacker 10.0.0.15 → victim 192.168.93.143
[+] Merging 15 attacker streams into one pcap...
[+] Verifying topology:




```
us@ubuntusur: ~
us@ubuntusur:~$ mkdir -p /tmp/mh_test
us@ubuntusur:~$ sudo suricata -r 15_vs_5_simulation.pcap -c /etc/suricata/suricata.yaml -l /tmp/mh_test/ -v
Notice: suricata: This is Suricata version 8.0.6 RELEASE running in USER mode
Info: cpu: CPUs/cores online: 8
Info: suricata: Setting engine mode to IDS mode by default
Info: exception-policy: master exception-policy set to: auto
Info: suricata: Preparing unexpected signal handling
Info: logopenfile: fast output device (regular) initialized: fast.log
Info: logopenfile: eve-log output device (regular) initialized: eve.json
Info: logopenfile: stats output device (regular) initialized: stats.log
Warning: detect: No rule files match the pattern /var/lib/suricata/rules/local.rules
Info: detect: 2 rule files processed. 137759 rules successfully loaded, 0 rules failed, 0 rules skipped
Info: threshold-config: Threshold config parsed: 0 rule(s) found
Info: detect: 137770 signatures processed. 17323 are IP-only rules, 5119 are inspecting packet payload, 114985 inspect application layer, 110 are decoder event only
Info: pcap: Starting file run for 15_vs_5_simulation.pcap
Notice: threads: Threads created -> RX: 1 W: 8 FM: 1 FR: 1   Engine started.
Info: checksum: No packets with invalid checksum, assuming checksum offloading is NOT used
Info: pcap: pcap file 15_vs_5_simulation.pcap end of file reached (pcap err code 0)
Notice: suricata: Signal Received.  Stopping engine.
Info: suricata: time elapsed 153.409s
Notice: pcap: read 1 file, 274320 packets, 124389045 bytes
Info: counters: Alerts: 13455
us@ubuntusur:~$ wc -l /tmp/mh_test/fast.log
13455 /tmp/mh_test/fast.log
us@ubuntusur:~$
```

The attacks are showing both on dashboard and also in logs:



SID	Formatted_Time	Severity	signature	src_ip	dest_ip	status
2101071	2026-08-12 02:05:49 PM	High	GPL WEB_SERVER .htpasswd access	10.0.0.1	192.168.93.139	new
2101071	2026-08-12 02:05:49 PM	High	GPL WEB_SERVER .htpasswd access	10.0.0.11	192.168.93.139	new
2101071	2026-08-12 02:05:49 PM	High	GPL WEB_SERVER .htpasswd access	10.0.0.12	192.168.93.140	new
2101071	2026-08-12 02:05:49 PM	High	GPL WEB_SERVER .htpasswd access	10.0.0.13	192.168.93.141	new
2101071	2026-08-12 02:05:49 PM	High	GPL WEB_SERVER .htpasswd access	10.0.0.14	192.168.93.142	new
2101071	2026-08-12 02:05:49 PM	High	GPL WEB_SERVER .htpasswd access	10.0.0.15	192.168.93.143	new
2101071	2026-08-12 02:05:49 PM	High	GPL WEB_SERVER .htpasswd access	10.0.0.2	192.168.93.140	new
2101071	2026-08-12 02:05:49 PM	High	GPL WEB_SERVER .htpasswd access	10.0.0.3	192.168.93.141	new
2101071	2026-08-12 02:05:49 PM	High	GPL WEB_SERVER .htpasswd access	10.0.0.4	192.168.93.142	new
2101071	2026-08-12 02:05:49 PM	High	GPL WEB_SERVER .htpasswd access	10.0.0.5	192.168.93.143	new
2101071	2026-08-12 02:05:49 PM	High	GPL WEB_SERVER .htpasswd access	10.0.0.6	192.168.93.139	new
2101071	2026-08-12 02:05:49 PM	High	GPL WEB_SERVER .htpasswd access	10.0.0.7	192.168.93.140	new
2101071	2026-08-12 02:05:49 PM	High	GPL WEB_SERVER .htpasswd access	10.0.0.8	192.168.93.141	new
2101071	2026-08-12 02:05:49 PM	High	GPL WEB_SERVER .htpasswd access	10.0.0.9	192.168.93.142	new
2101071	2026-08-12 02:05:49 PM	High	GPL WEB_SERVER .htpasswd access	10.0.0.10	192.168.93.143	new
2610370	2026-08-12 02:05:49 PM	Medium	TGI HUNT .bin HTTP download missing headers	10.0.0.1	192.168.93.139	new
2610370	2026-08-12 02:05:49 PM	Medium	TGI HUNT .bin HTTP download missing headers	10.0.0.11	192.168.93.139	new
2610370	2026-08-12 02:05:49 PM	Medium	TGI HUNT .bin HTTP download missing headers	10.0.0.12	192.168.93.140	new
2610370	2026-08-12 02:05:49 PM	Medium	TGI HUNT .bin HTTP download missing headers	10.0.0.13	192.168.93.141	new
2610370	2026-08-12 02:05:49 PM	Medium	TGI HUNT .bin HTTP download missing headers	10.0.0.14	192.168.93.142	new
2610370	2026-08-12 02:05:49 PM	Medium	TGI HUNT .bin HTTP download missing headers	10.0.0.15	192.168.93.143	new
2610370	2026-08-12 02:05:49 PM	Medium	TGI HUNT .bin HTTP download missing headers	10.0.0.2	192.168.93.140	new
2610370	2026-08-12 02:05:49 PM	Medium	TGI HUNT .bin HTTP download missing headers	10.0.0.3	192.168.93.141	new

```
us@ubuntusur:~$ sudo tail -f /var/log/suricata/eve.json | jq 'select(.event_type=="alert") | {sig: .alert.signature, src: .src_ip, dst: .dest_ip}'
{
  "sig": "ET INFO Possible Kali Linux hostname in DHCP Request Packet",
  "src": "192.168.93.130",
  "dst": "192.168.93.254"
}
{
  "sig": "SURICATA HTTP unable to match response to request",
  "src": "192.168.93.142",
  "dst": "10.0.0.9"
}
{
  "sig": "SURICATA HTTP unable to match response to request",
  "src": "192.168.93.139",
  "dst": "10.0.0.6"
}
{
  "sig": "GPL EXPLOIT .htr access",
  "src": "10.0.0.10",
  "dst": "192.168.93.143"
}
{
  "sig": "GPL EXPLOIT .htr access",
  "src": "10.0.0.9",
  "dst": "192.168.93.142"
}
{
  "sig": "GPL EXPLOIT .htr access",
  "src": "10.0.0.8",
  "dst": "192.168.93.141"
}
{
  "sig": "GPL EXPLOIT .htr access",
  "src": "10.0.0.7",
  "dst": "192.168.93.140"
}
{
  "sig": "GPL EXPLOIT .htr access"
```